

■ 2.8.3 Advanced Topic: Stack Space

On most computer systems, the memory used by a running program is divided into two parts: memory explicitly allocated by the program, and “stack space”. Every time an internal routine is called in the program, a certain amount of stack space is used to store parameters associated with the call. On many computer systems, the maximum amount of stack space that can be used by a program must be specified in advance. If the specified stack space limit is exceeded, the program usually just exits.

Results and intermediate expressions are all stored in memory that is explicitly allocated by *Mathematica*. However, calls of internal *Mathematica* routines use up stack space. The amount of stack space they use can become large if the routines are called recursively many times.

On most computer systems, it is difficult to control or even monitor directly the stack space used in a *Mathematica* session. Nevertheless, there is one important way that you can exercise somewhat indirect control over the amount of stack space used.

<code>\$RecursionLimit</code>	the number of levels of recursion now allowed
<code>\$RecursionLimit=<i>n</i></code>	set the number of levels of recursion allowed to <i>n</i>

Controlling the number of levels of recursion allowed during evaluation.

One of the primary sources of recursion in *Mathematica* is the evaluation of functions which are defined in terms of each other. The global variable `$RecursionLimit` specifies the number of levels of recursion that are allowed in the evaluation process. You should try to make sure that the limits you set are low enough that *Mathematica* should not run out of stack space simply as a result of doing recursive evaluations.